

AXLERO SOLUTIONS

PROJECT DOCUMENT – ADVANCED DATA ANALYTICS

Real-World Applications - Hands-on System Building

As enterprise data architecture matures into 2026, the focus is rapidly shifting from passive dashboards to governed, AI-driven action [1.1.1, 1.1.8]. This third volume of advanced data analytics projects focuses on the absolute forefront of the modern data stack: **Agentic BI via Semantic Layers** [1.1.6], **Real-Time Lakehouse Observability** [1.1.4], and **Closed-Loop Prescriptive Analytics** [1.1.1].

Here are three highly advanced, enterprise-grade Data Analytics architectures for a senior-level portfolio.

Project 1 - "MetricMind": Agentic Semantic BI Engine

Domain: Enterprise Analytics & Agentic AI

Problem Statement: Giving an LLM raw access to a data warehouse to perform "Text-to-SQL" usually ends in disaster. The AI hallucinates joins, ignores business logic, and generates revenue metrics that differ wildly from official financial reports, creating a "deluge of distrust" [1.1.6, 1.1.10].

Use Case: An executive asks the MetricMind chat interface, *"Why did our European margins drop last quarter?"* Instead of the AI writing rogue SQL against raw tables, the agent interfaces strictly with an enterprise **Semantic Layer** (e.g., Cube or dbt Semantic Layer). The Semantic Layer defines the exact mathematical formula for "margin" and "quarter." The LLM acts as an orchestrator, retrieving the governed metric definitions, compiling the correct query, and presenting a multi-step analytical explanation. Finance and Sales finally see the exact same numbers [1.1.6].

Key Modules:

- **Semantic Layer (Cube.dev or dbt):** The central repository where all business metrics (Revenue, Churn, Margin) are mathematically defined as code, decoupling definitions from the BI tool [1.1.6].
- **Agentic Orchestrator (LangChain / Llama 3):** The AI agent that translates natural language into semantic API calls rather than raw SQL dialects.
- **Data Lakehouse (Snowflake / Databricks):** The underlying storage and compute engine serving the analytical queries.
- **Conversational BI Interface (Next.js & Tremor):** A custom chat interface that renders natural language text alongside structured, interactive charts (ECharts).

Week-wise Development Plan:

	Data Modeling & Semantics (dbt, Cube.dev)	Agentic AI & UI (LangChain, Next.js)
	Warehouse & Modeling: Ingest mock corporate data into Snowflake. Build the raw and transformed layers using standard dbt models.	Chat Scaffolding: Build the Next.js chat interface. Implement a streaming text response UI.
	Semantic Layer Definition: Define the exact dimensions (Time, Geography) and measures (Revenue, Cost) in Cube.dev YAML files. Ensure the API returns strict JSON data.	Agent Prompts: Write the LangChain orchestration logic. Give the LLM access to the Cube.dev schema so it understands exactly which metrics are available to query.
	Governance Audit: Prove that asking the system for "Q3 Revenue" returns the exact same numerical result every single time without SQL hallucinations.	API Check: Ensure the LLM successfully translates "Show me European sales" into the correct REST API payload for the Semantic Layer.
	Multi-Step Reasoning: Upgrade the agent. If margins dropped, the agent should automatically query a secondary breakdown (e.g., shipping costs vs. material costs) to find the root cause [1.1.1].	Dynamic Visualizations: Parse the JSON data returned to the chat UI and dynamically render the correct chart type (e.g., line chart for time-series, bar chart for categorical).
	Cost Governance: Implement query limits. Prevent the AI agent from executing unbounded, expensive exploratory queries on the warehouse [1.1.6].	Refine & Polish: Add "View SQL / View API Call" buttons to the UI to provide complete transparency into how the AI fetched the data.
	A masterclass in bridging the gap between Generative AI and strict corporate data governance.	A production-ready Conversational BI tool that finance teams can actually trust.

Project 2 - "Ice Stream": Real-Time Lakehouse Observability

Domain: Data Architecture & Reliability Engineering

Problem Statement: Traditional batch ETL jobs introduce hours of delay. When bad data (e.g., null values or schema drift) enters the pipeline, it often isn't caught until a dashboard breaks during a board meeting. Real-time data quality monitoring is required to prevent "garbage in, garbage out" at scale [1.1.4].

Use Case: A massive e-commerce platform uses IceStream to monitor checkout telemetry. Streaming data flows through Apache Kafka directly into an Apache Iceberg table (a modern open table format providing ACID transactions on data lakes) [1.1.4]. Apache Flink continuously monitors this stream. If it detects an anomaly (e.g., the "Tax Amount" column suddenly contains 50% NULL values), it doesn't just send a Slack alert; it automatically pauses the downstream analytical pipeline, quarantines the bad data, and initiates a re-fetch, requiring zero human intervention [1.1.1].

Key Modules:

- **Streaming Engine (Apache Flink & Kafka):** Handles the high-velocity ingestion and real-time stateful processing of telemetry data.
- **Open Table Format (Apache Iceberg):** Allows for fast, ACID-compliant UPSERTs and time-travel queries directly on top of cheap cloud object storage (S3), replacing traditional rigid data warehouses [1.1.4].
- **Observability Rules Engine (Python / Great Expectations):** Defines the strict data quality thresholds and anomaly detection heuristics [1.1.2].
- **Data Lineage Dashboard (React & React Flow):** A visual UI mapping the flow of data and highlighting exactly which node in the pipeline is currently quarantined.

Week-wise Development Plan:

Week	Lakehouse Engineering (Kafka, Flink, Iceberg)	Observability & Automation (Python, React)
Week 1	Stream Generation: Build a Python script generating thousands of mock e-commerce transactions per second, deliberately injecting	Lineage UI: Build a Node-based React Flow dashboard to visualize the data pipeline (Ingest → Process → Serve).

Week	Lakehouse Engineering (Kafka, Flink, Iceberg)	Observability & Automation (Python, React)
	occasional null values and schema changes.	
Week 2	Lakehouse Foundation: Configure an Apache Iceberg catalog. Write Flink jobs to consume the Kafka stream and continuously append records to the Iceberg tables in real-time.	Rules Definition: Implement data quality assertions (e.g., <code>expect_column_values_to_not_be_null</code>) using Great Expectations or a custom Python engine.
Mid-Project Review	ACID Audit: Prove that Iceberg successfully handles concurrent real-time writes and reads without locking or corrupting the data lake [1.1.4].	Detection Check: Ensure the system immediately flags when the injected bad data hits the Flink processor.
Week 3	Automated Remediation: Write the "Circuit Breaker" logic. If the error rate exceeds 2%, Flink routes the incoming stream to a "Dead Letter Queue" (DLQ) Iceberg table instead of the main table [1.1.1].	Live Alerts: Connect the circuit breaker status to the React Flow UI via WebSockets, instantly turning the affected pipeline node red.
Week 4	Time Travel Queries: Utilize Iceberg's snapshot isolation to query the exact state of the data <i>before</i> the anomaly occurred, demonstrating easy rollback capabilities.	Refine & Polish: Build a detailed incident log in the UI showing exactly why a pipeline was paused and when it resumed.

Week	Lakehouse Engineering (Kafka, Flink, Iceberg)	Observability & Automation (Python, React)
Final Review	An advanced deployment of the 2026 industry-standard "Lakehouse" architecture.	A self-healing data pipeline that treats data quality as a proactive engineering discipline.

Project 3 - "Supply Prescript": Closed-Loop Prescriptive Analytics

Domain: Supply Chain Operations & Operations Research

Problem Statement: Predictive analytics (e.g., predicting a supply chain delay) are now standard, but they only tell you what *will* happen. The human operator still has to figure out *what to do*. Furthermore, standard dashboards don't learn; if an operator makes a decision, the system rarely tracks whether that decision actually worked [1.1.1].

Use Case: A logistics manager is warned by the predictive model of an impending 14-day delay for microchips. Instead of stopping there, the SupplyPrescript engine runs a linear optimization algorithm and *prescribes* three mathematically optimal alternatives: (A) Pay \$15k for Air Freight, (B) Buy from a secondary supplier at a 10% premium, or (C) Delay the final product launch. The manager clicks Option A directly in the dashboard. The system writes this decision back to the operational database. Three weeks later, the system evaluates the outcome—learning that air freight actually cost \$18k—and adjusts its future optimization weights accordingly (Closed-Loop Analytics) [1.1.1].

Key Modules:

- **Predictive Model (XGBoost / LightGBM):** Predicts the probability and duration of a supply chain disruption based on historical lead times.
- **Prescriptive Solver (SciPy / PuLP):** A mathematical optimization engine that calculates the absolute best business decisions subject to constraints (Budget, Time, Capacity).
- **Write-Back Architecture (Snowflake & FastAPI):** A transactional pipeline allowing business users to insert operational decisions *back* into the data warehouse, closing the loop.
- **Operational Dashboard (Retool or React):** An interactive interface where users review AI recommendations and execute decisions directly, replacing passive read-only BI reports [1.1.1].

Week-wise Development Plan:

Week	ML & Optimization Engine (XGBoost, SciPy)	Operational UI & Write-Back (FastAPI, Retool)
Week 1	Predictive Baseline: Train an XGBoost model on historical mock supply chain data to accurately predict shipment delays (Classification/Regression).	App Scaffolding: Set up a Retool app (or custom React UI) connected to a PostgreSQL or Snowflake database.
Week 2	Mathematical Optimization: Define the business constraints (e.g., maximum budget, minimum inventory). Write a SciPy Linear Programming script to generate the 3 best alternative actions when a delay is predicted.	Prescriptive UI: Build the UI cards displaying the model's prescriptions, clearly highlighting the trade-offs (Cost vs. Speed) for each option.
Mid-Project Review	Optimization Audit: Prove that the SciPy solver never recommends an action that violates the hard budget constraints defined in the code.	Write-Back Check: Ensure that clicking an "Execute Decision" button in the UI successfully performs an INSERT statement back into the operational database.
Week 3	The Closed Loop: Write the evaluation script. Compare the <i>predicted</i> cost of the user's decision against the <i>actual</i> historical outcome stored in the database [1.1.1].	Feedback UI: Build an analytics view showing the "Decision ROI"—tracking how often the AI's recommendations resulted in positive business outcomes.
Week 4	Continuous Learning: Implement a pipeline where the discrepancies discovered in the Closed Loop automatically trigger a re-training of the XGBoost model.	Refine & Polish: Polish the workflow, ensuring the analyst is no longer just a passive observer, but an active participant in an AI-driven workflow [1.1.1].

Week	ML & Optimization Engine (XGBoost, SciPy)	Operational UI & Write-Back (FastAPI, Retool)
Final Review	A massive leap from "What will happen?" to "What should we do?" using Operations Research.	A true closed-loop operational application that compounds business value over time.

Common Features Across All Projects

- **Agentic & Automated Workflows:** Replacing manual, human-in-the-loop BI analysis with agentic orchestration, self-healing data pipelines, and prescribed decision-making [1.1.1, 1.1.6].
- **Modern Data Architecture (2026 Standards):** Utilizing Lakehouse architectures (Apache Iceberg/Delta Lake), semantic layers, and streaming ingestions to overcome the latency and governance limitations of legacy data warehouses [1.1.4, 1.1.6].
- **Write-Back and Closed-Loop Systems:** Moving away from read-only "dead" dashboards. These projects involve applications where data flows bidirectionally, allowing user decisions to mutate the underlying databases and retrain models [1.1.1].
- **Strict Governance & Observability:** Treating data quality and metric consistency as engineering requirements (using Circuit Breakers, Great Expectations, and Semantic logic) rather than business afterthoughts [1.1.2, 1.1.8].

Overall Project Summary

Project	Target Analytics Domain	Technology Focus
"MetricMind": Semantic BI Agent	Agentic AI & BI Governance	Semantic Layers (Cube.dev/dbt), LangChain, LLMs, Next.js
"IceStream": Lakehouse Observability	Real-Time Data Architecture	Apache Iceberg, Flink, Kafka, Great Expectations, React Flow
"SupplyPrescript": Closed-Loop Analytics	Operations Research	SciPy Optimization, XGBoost, Write-Back APIs, Retool/React